INEM · IA pour les systèmes embarqués · 2026–2027

Construire, entraîner et optimiser un petit réseau de prédiction sur système embarqué

Un réseau à mélange temporel et de variables, inspiré de TSMixer

Travaux dirigés et initiation à la recherche expérimentale · Roland Cédric TAYO

Résumé

Ce projet consiste à programmer un petit réseau de prédiction sans bibliothèque d'apprentissage. À partir de trois signaux simulés, le réseau utilise les seize derniers instants pour prédire une valeur future. Les étudiants écrivent les calculs, les gradients et l'apprentissage, puis déploient leur propre inférence sur Raspberry Pi. Ils étudient ensuite une optimisation d'une projection dense, vérifient la correction et mesurent son intérêt matériel. Le travail produit des courbes, des traces reproductibles et un rapport court au format scientifique. Aucun code ni environnement prêt à l'emploi n'est fourni ; les équations, les contraintes et les points de vérification constituent le contrat commun.

Mots-clés : prédiction ; réseau résiduel ; calcul embarqué ; mémoire ; optimisation ; reproductibilité.

1. Problème et objectifs

Un appareil doit prédire localement une mesure à partir de son historique, sans dépendre d'Internet. Sa mémoire et son temps de calcul sont limités. L'objectif est de comprendre comment un modèle apprend, puis comment son implémentation peut être adaptée à une petite cible. Les signaux du TD sont synthétiques : ils servent à vérifier une méthode, sans démontrer une application industrielle.



Figure 1. Le même modèle est appris sur PC, puis exécuté et mesuré sur la cible.

Question de recherche. À calcul et poids fixés, quelle organisation d'une projection dense réduit le temps de calcul ou la mémoire temporaire sur la cible, et dans quelles conditions cet avantage disparaît-il ?

Acquis attendus

- Expliquer poids, biais, activation, perte, gradient, apprentissage et inférence.
- Programmer et vérifier un réseau, séparer les données et interpréter ses erreurs.
- Déployer une inférence autonome ; distinguer poids, buffers, stockage et RAM du processus.
- Formuler une hypothèse, mesurer un compromis et rendre l'expérience reproductible.

Cadre. Quatre journées : 14 h de cours et 14 h de TD, puis approfondissement jusqu'en décembre selon le calendrier annoncé. Travail en équipes de 4 à 5 ; maîtrise individuelle évaluée. Prérequis : fonctions, boucles, tableaux, compilation, terminal, Git, produits de matrices et dérivées simples. Le storyboard de 3 h est une introduction ; les 14 h de cours et le travail personnel accompagnent les calculs ci-dessous.

Rappels A. Lire les données et les équations

Une équation est une recette. On remplace chaque symbole par sa valeur, puis on calcule. Ici, les signaux n'ont pas d'unité physique imposée. Un instant est un pas d'échantillonnage, pas nécessairement une seconde.

Symbole	Sens dans ce TD
s ; t	s est l'identifiant de séquence, de 0 à 23. t est l'indice de temps : 0 à 191 lors de la génération. t−1 signifie « l'instant précédent ».
x_1, x_2, x_3	Les trois valeurs observées à chaque instant. $x_c(t)$ est la valeur de la variable numéro c à l'instant t ; c vaut 1, 2 ou 3.
P ; φ ; π	P est la période entière tirée pour une séquence, entre 24 et 64. φ (phi) est un décalage de phase, en radians. π (pi) ≈ 3,14159. La période n'est pas un nombre de paramètres du réseau.
$\varepsilon_c(t)$	ε (epsilon) désigne un petit bruit aléatoire pour la variable c à l'instant t. Un nouveau tirage uniforme dans [−0,02 ; 0,02] est effectué pour chaque variable et instant.
sin ; cos ; max	Sinus et cosinus prennent ici un angle en radians. max(a,b) conserve la plus grande des deux valeurs. Ainsi max(0,−2)=0 et max(0,3)=3.
μ_c ; σ_c	μ (mu) est la moyenne de la variable c sur les observations d'apprentissage. σ (sigma) est son écart-type : il décrit l'amplitude de ses variations autour de cette moyenne.
N_{obs} ; r	N_{obs}=16×160=2 560 observations d'apprentissage par variable, avant fenêtrage. r parcourt ces observations, sans répéter les points présents dans plusieurs fenêtres.

$$\mu_c = (1/N_{obs}) \Sigma_{r=1}^{Nobs} x_{c,r},$$

$$\sigma_c = \sqrt{[(1/N_{obs}) \Sigma_{r=1}^{Nobs} (x_{c,r}-\mu_c)^2]},$$

$$\bar{\sigma}_c = \max(\sigma_c, 10^{-6}) ; \quad \tilde{x}_c = (x_c-\mu_c)/\bar{\sigma}_c.$$

Σ (sigma majuscule) signifie « additionner tous les termes ». √ est la racine carrée. Le carré ² multiplie une quantité par elle-même. La barre sur σ indique l'écart-type protégé contre une division par zéro ; le tilde sur x indique une valeur normalisée. Cette recette utilise le diviseur N_{obs}, et non $N_{obs}-1$.

La cible y est le prochain x_1. On entraîne avec $\tilde{y}=(y-\mu_1)/\bar{\sigma}_1$. La sortie normalisée $\tilde{\hat{y}}$ redevient une prédiction lisible par $\hat{y}=\bar{\sigma}_1\tilde{\hat{y}}+\mu_1$. Un chapeau signifie toujours « prédit » ; les métriques finales comparent $\hat{y}$ et y sur l'échelle d'origine.

Erreurs de prédiction. MSE est l'erreur quadratique moyenne ; MAE, l'erreur absolue moyenne. Dans les formules de la section 2, N est le nombre de prédictions évaluées, i va de 1 à N, y_i est la cible et $\hat{y}_i$ sa prédiction. |a| donne la valeur absolue de a. On moyenne des erreurs, pas des pourcentages de bonnes classes.

2. Données : générer, séparer, prédire

Écrire un générateur de **24 séquences indépendantes**. Pour chaque séquence, tirer une période entière P entre 24 et 64 et une phase φ dans $[0, 2\pi[$. Initialiser les états précédents à zéro. Pour $t = 0,\ldots,191$, calculer :

$$x_1(t) = 0{,}60\, x_1(t-1) + 0{,}25\, x_2(t-1) + 0{,}15 \max(0, x_3(t-1)) + \varepsilon_1(t),$$

$$x_2(t) = \sin(2\pi t / P + \varphi) + \varepsilon_2(t),$$

$$x_3(t) = 0{,}70\, x_3(t-1) + 0{,}30 \cos(2\pi t / (P+7) + \varphi) + \varepsilon_3(t).$$

Les bruits ε sont indépendants et uniformes entre $-0{,}02$ et $+0{,}02$. Utiliser la graine **17 + identifiant de séquence**, de 0 à 23 ; nommer le générateur pseudo-aléatoire utilisé. Écarter les 32 premiers instants de chaque séquence et conserver les 160 suivants. Exporter un CSV par séquence : `t,x1,x2,x3`. Des langages différents peuvent produire des tirages différents : toute comparaison utilise exactement les mêmes CSV, dont les empreintes sont conservées.

2.1. Partition avant fenêtrage

Ensemble	Séquences	Usage
Apprentissage	0 à 15	Poids et statistiques de normalisation
Validation	16 à 19	Réglages et sélection du checkpoint
Test réservé	20 à 23	Évaluation finale après gel des choix

Créer ensuite les fenêtres à l'intérieur de chaque séquence. Une entrée contient 16 instants × 3 variables ; sa cible est x_1 à l'instant suivant. Il y a **144 exemples par séquence**, soit 2 304 exemples train, 576 validation et 576 test. Aucune fenêtre ne traverse deux séquences ou deux ensembles.

2.2. Normalisation et références

Calculer moyenne μ_c et écart-type σ_c sur les observations train de chaque variable ; remplacer σ par $\max(\sigma, 10^{-6})$. Normaliser les entrées avec ces seules statistiques. Normaliser la cible avec μ_1, σ_1 ; dénormaliser les prédictions pour les figures et les métriques finales.

Comparer obligatoirement le réseau à deux prédicteurs : **persistance**, soit la dernière valeur observée de x_1, et **régression linéaire** sur les 48 valeurs d'entrée. La seconde se programme avec la même perte et une descente de gradient, sans solveur externe.

$$\mathrm{MSE} = (1/N)\, \Sigma_i(\hat{y}_i - y_i)^2 \; ; \quad \mathrm{MAE} = (1/N)\, \Sigma_i|\hat{y}_i - y_i|.$$

À montrer. Tracer les trois signaux d'une séquence, puis valeurs réelles et prédites. Rapporter MSE et MAE sur l'échelle d'origine : ce problème de régression n'a pas un « taux de bonnes classes ». Une référence simple peut être meilleure que le réseau. Les équations du simulateur rendent aussi possible un prédicteur analytique ; discuter cette limite plutôt que revendiquer un besoin industriel d'IA.

3. Réseau à programmer

Le réseau conserve l'idée de mélange temporel et de variables de TSMixer [1,2]. Cette **adaptation pédagogique** comporte un seul bloc, sans normalisation interne, dropout ni attention. Elle ne reproduit pas l'architecture complète ou les performances de l'article.

3.1. Propagation avant

Noter $X \in \mathbb{R}^{16\times 3}$ la fenêtre normalisée et ReLU(a) = max(0,a), élément par élément. Pour chaque variable *c*, utiliser les **mêmes** matrices temporelles A et B :

$$h_c = \mathrm{ReLU}(A\,X_{:,c} + a)\ ; \quad U_{:,c} = X_{:,c} + B\,h_c + b.$$

Pour chaque instant *t*, utiliser les **mêmes** matrices C et D :

$$g_t = \mathrm{ReLU}(C\,U_{t,:}^{\top} + q)\ ; \quad Z_{t,:}^{\top} = U_{t,:}^{\top} + D\,g_t + d,$$

$$\hat{y} = v^{\top}\,\mathrm{vec}(Z) + \beta.$$

vec parcourt les instants, puis les variables. Les additions avec X et U sont les connexions résiduelles : elles transmettent l'entrée et une correction apprise.

Paramètres	Dimensions	Nombre
A, a ; B, b	8×16, 8 ; 16×8, 16	280
C, q ; D, d	8×3, 8 ; 3×8, 3	59
v, β	48, 1	49
Total	**388 paramètres entraînables**	**1 552 octets en float32**

3.2. Gradients et apprentissage

Écrire la propagation arrière par la règle de la chaîne. Pour une couche y = Wx+b et un gradient reçu δ : $\partial L/\partial W = \delta x^{\top}$, $\partial L/\partial b = \delta$ et $\partial L/\partial x = W^{\top}\delta$. ReLU transmet le gradient si son entrée est positive ; prendre une dérivée nulle en zéro. Une addition transmet le gradient aux deux branches. **Les paramètres partagés cumulent les contributions de tous leurs usages.**

Initialiser les poids uniformément entre −0,1 et +0,1 et les biais à zéro. Point de départ : mini-lots de 16, perte quadratique moyenne, descente de gradient avec $\eta = 10^{-3}$, au plus 20 époques. Remettre les accumulateurs de gradients à zéro à chaque lot. Le gradient de sortie vaut $2(\hat{y}-y)/m$ pour un lot de m exemples. Appliquer $\theta \leftarrow \theta - \eta\nabla L$ après le lot. Documenter tout réglage choisi sur validation ; garder le meilleur checkpoint de validation.

Vérifications. Tester une projection à la main ; contrôler les dimensions ; comparer des gradients analytiques et des différences finies centrées [3], en double précision sur quelques entrées, hors changement de branche de ReLU. Utiliser $g_{num} = [L(\theta+h)-L(\theta-h)]/(2h)$, avec $h = 10^{-5}$. Viser $|g-g_{num}| \le 10^{-6} + 10^{-3}\max(|g|,|g_{num}|)$. Contrôler deux coefficients par matrice et vecteur de biais, ainsi que β, sur deux petites entrées. Puis vérifier une forte baisse de perte sur un minuscule sous-ensemble.

Chronométrer deux époques avant le lancement complet. Un entraînement de quelques minutes est une cible de dimensionnement, pas une durée garantie selon le langage et le PC. Conserver durée, perte train et MSE validation à chaque époque ; signaler un blocage plutôt que remplacer silencieusement le modèle.

Rappels B. Lire le réseau et l'apprentissage

Symbole	Signification et dimensions
X ; U ; Z	Trois tableaux de 16 lignes (instants) et 3 colonnes (variables). X est l'entrée ; U est le résultat du mélange temporel ; Z est le résultat du mélange entre variables.
$\mathbb{R}$; ∈ ; ×	$\mathbb{R}$ désigne les nombres réels ; ∈ signifie « appartient à ». $X \in \mathbb{R}^{16\times 3}$ signifie « tableau réel de 16 lignes et 3 colonnes ». × sépare ici les dimensions.
A, a ; B, b	Poids et biais du mélange temporel : A de taille 8×16, a de taille 8 ; B de taille 16×8, b de taille 16. Les minuscules sont des vecteurs de biais.
C, q ; D, d	Poids et biais du mélange des variables : C de taille 8×3, q de taille 8 ; D de taille 3×8, d de taille 3. Le biais q est distinct de l'indice c, qui est un numéro de variable.
h_c ; g_t	Vecteurs cachés de 8 valeurs : h_c mélange les 16 instants de la variable c ; g_t mélange les 3 variables à l'instant t.
: ; ⊤ ; vec	« : » sélectionne toutes les lignes ou colonnes. ⊤ transpose un tableau (échange lignes et colonnes). vec(Z) range les 48 valeurs : $Z_{1,1}$, $Z_{1,2}$, $Z_{1,3}$, $Z_{2,1}$, etc.
v ; β ; $\tilde{y}$	v contient les 48 poids de sortie ; β (bêta) est le biais scalaire. $\tilde{y}$ est la valeur prédite normalisée. Dans les équations du réseau ci-contre, ŷ désigne cette sortie normalisée pour alléger l'écriture.
ReLU	Activation max(0,a), appliquée à chaque valeur. La lettre a dans cette définition représente une valeur quelconque, pas seulement le biais temporel.
θ ; L ; ∇ ; ∂	θ (thêta) regroupe les 388 paramètres. L est la perte. ∇L (nabla L) regroupe les dérivées de cette perte ; ∂ désigne une dérivée par rapport à une variable. Une dérivée indique comment une petite variation change la perte.
η ; δ ; W, x, y, b	η (êta) est le pas d'apprentissage. δ (delta) est le gradient reçu par une couche. Dans y=Wx+b, W, x, y, b sont les poids, l'entrée, la sortie et les biais d'une couche quelconque ; y n'est alors pas la cible du jeu de données.

Convention de lot. m=16 est le nombre d'exemples du mini-lot (éventuellement moins pour le dernier). Pour $L=(1/m)\Sigma_{i=1}^{m}(\tilde{y}_i-\tilde{y}_i)^2$, le gradient de sortie est $2(\tilde{y}_i-\tilde{y}_i)/m$. Additionner ensuite toutes les contributions aux paramètres partagés. **Ne pas diviser une seconde fois par m.** $\theta \leftarrow \theta-\eta\nabla L$ signifie « remplacer les poids par leurs valeurs corrigées ». Une époque est un passage sur tous les exemples d'apprentissage.

Contrôle numérique. $h=10^{-5}$ est une petite perturbation d'un seul coefficient de θ ; ce h n'est pas le vecteur h_c. g est sa dérivée analytique ; g_{num} est son approximation numérique. |a| est la valeur absolue de a : |−3|=3. Le signe ≤ signifie « inférieur ou égal » ; 10^{-6}=0,000001.

Temps et accélération. $T_{réf}$ et T_{var} sont les coûts de référence et de variante, strictement positifs et mesurés dans la même unité (secondes, millisecondes ou nanosecondes). Une accélération de 2 signifie deux fois moins de temps ; un gain de 50 % signifie une moitié du temps économisée. Une durée nulle à cause d'un timer trop grossier ne permet aucun ratio.

4. Réalisation et déploiement

Le groupe choisit son langage et construit son environnement. Organiser le programme en fonctions testables : génération, normalisation, projection, ReLU, bloc résiduel, perte, gradients, mise à jour, sauvegarde, inférence et mesures. Aucun framework n'est nécessaire. Les données, poids et configurations doivent pouvoir être relus indépendamment du programme d'apprentissage.

4.1. Quatre TD de 3 h 30

Séance	Travail et temps indicatifs	Preuve à conserver
TD1 Fondations	25 min : besoin et dimensions ; 40 : données et partition ; 70 : projection, activation et gradients ; 45 : tests ; 30 : dépôt et synthèse.	CSV, exemple manuel et contrôle numérique de gradient.
TD2 Apprendre	70 min : réseau et gradients partagés ; 60 : entraînement ; 40 : références et erreurs ; 25 : sauvegarde/rechargement ; 15 : bilan.	Poids, courbes, MSE des références et du réseau.
TD3 Embarquer	40 min : carte et compilation ; 55 : inférence ; 35 : vérification PC/cible ; 50 : mesures de référence ; 30 : hypothèse.	Inférence correcte sur carte, ressources et protocole daté.
TD4 Enquêter	60 min : une variante ; 35 : correction ; 45 : mesures ; 30 : effet global ; 25 : reproduction croisée ; 15 : conclusion.	Comparaison pilote, figure et prochaine expérience.

Les cours accompagnent les calculs, les contraintes matérielles et la méthode expérimentale. En fin de TD4, une première expérience suffit ; la campagne approfondie et le rapport se poursuivent à domicile. L'autonomie sur les outils suppose les prérequis : tester compilation et transfert avant la séance évite que les installations consomment tout le TD.

4.2. Contrat matériel

Cible du TD : Raspberry Pi sous Linux. Pour environ 35 étudiants et 14 Pi, prévoir sept groupes de cinq, un Pi par groupe au minimum. Il faut une alimentation adaptée, une microSD avec Raspberry Pi OS et un accès réseau/SSH ou écran/clavier. Avant TD1, vérifier démarrage, transfert et exécution. Noter modèle, RAM, architecture 32/64 bits, OS, CPU, compilateur et options. Python sur PC puis C/C++ sur Pi est conseillé ; tout autre choix compatible doit être vérifié.

Transférer code, poids, statistiques train et fenêtres de test par Git, SCP ou clé USB. Compiler et exécuter sur le Pi : il calcule seul, sans service cloud. SSH sert à le piloter et récupérer les résultats, pas à calculer sur PC. Aucun capteur ni accélérateur n'est requis. Le réseau est petit : il ne met pas la RAM du Pi en difficulté.

Correction. Comparer les mêmes entrées sur PC et carte avant d'annoncer une performance. Pour la référence flottante, cible indicative : $|\hat{y}_{\text{carte}} - \hat{y}_{\text{PC}}| \leq 10^{-5} + 10^{-4}|\hat{y}_{\text{PC}}|$ sur les sorties normalisées ; expliquer et justifier tout ajustement. Sauvegarder puis recharger ne doit pas changer les prédictions.

5. Petite recherche : améliorer une brique

Socle commun. Étudier la projection dense y = Wx+b, réellement appelée par le réseau. Partir d'une référence scalaire correcte, compilée avec les mêmes options que la variante. Écrire avant les essais : « Nous pensons que […] réduira […], car […]. Cette hypothèse sera mise en défaut si […]. » Une seule hypothèse principale et trois variantes au maximum.

Levier possible	Modification étudiée	Contrepartie à rechercher
Parcours	Changer l'ordre des boucles ou la disposition des poids.	Préparation et lectures supplémentaires.
Réemploi	Calculer plusieurs sorties ensemble ; essayer un petit bloc.	Registres, taille du code, cas trop petits.
Buffers	Éviter des copies ou réutiliser un tampon dont la durée de vie est terminée.	Écrasement de données encore nécessaires.

Conserver les mêmes poids : ces pistes ne demandent pas de réentraînement. L'étude d'une précision réduite est une extension possible après accord de périmètre : elle change le calcul et impose aussi de mesurer la MSE/MAE finale. Elle n'est pas obligatoire et n'accorde pas de bonus automatique.

5.1. De la brique au réseau

Mesurer les formes du modèle, puis compléter le banc isolé par des projections 16→8, 32→16 et 64→32, avec des valeurs déterministes et sauvegardées. Les deux dernières formes explorent une frontière ; elles ne constituent pas de nouveaux réseaux entraînés. Si les poids sont réorganisés, chronométrer séparément leur préparation et son amortissement.

Vérifier les sorties de la brique, puis exécuter le réseau complet avec la fonction modifiée. La sélection à l'exécution ou à la compilation doit montrer quelle version est appelée. Chercher une condition favorable et une condition défavorable ; leur existence n'est pas garantie. Si les gains restent dans la dispersion ou si la référence simple gagne, le rapport doit le dire.

5.2. Protocole commun

- Figer données, poids, versions, options et périmètre. Comparer dans le même langage sur le même Pi. Noter fréquence, charge, température et refroidissement. Séparer préparation, inférence et fichiers.
- Utiliser clock_gettime(CLOCK_MONOTONIC) en C/C++ ou time.perf_counter_ns() en Python. Échauffer puis mesurer des lots assez longs ; garder un checksum après la mesure. Exclure fichiers et affichage. Une unité en ns ne garantit pas une précision de 1 ns.
- Faire au moins trois campagnes en alternant référence et variante. Garder durée du lot et nombre d'appels ; calculer le temps moyen par appel, puis médiane et dispersion entre lots. Linux varie : aucun temps réel garanti n'est démontré.
- Séparer octets des poids, buffers, fichiers et RAM du processus. Un pic RSS inclut le runtime, pas seulement les tableaux ; indiquer outil et unité. Pas de mesure d'énergie sans instrument adapté.

$$\text{Accélération} = T_{\text{réf}} / T_{\text{var}} \; ; \quad \text{gain de temps (\%)} = 100 \times (T_{\text{réf}} - T_{\text{var}}) / T_{\text{réf}}.$$

Afficher les octets en plus des pourcentages mémoire. Une amélioration sur PC ne prouve pas un gain sur carte. Une comparaison facultative à une bibliothèque optimisée [4] situe le résultat ; dépasser sa propre version initiale n'équivaut pas à dépasser l'état de l'art.

6. Règles, rendus et évaluation

Autorisé	Non autorisé dans le socle
Bibliothèque standard : fichiers, mathématiques, allocation, temps, aléatoire ; outils système et de compilation du Pi.	Framework de réseau, différentiation automatique, BLAS ou fonction externe réalisant à votre place projection, gradients, apprentissage ou inférence.
Outils de figures et de mise en page ; NumPy limité à l'analyse et aux figures ; tests écrits par l'équipe.	Utiliser NumPy ou un équivalent pour les calculs du modèle évalué ; importer un modèle ou moteur existant à la place de son implémentation.
Documentation, articles, petits extraits génériques attribués ; bibliothèque optimisée dans une expérience de comparaison clairement séparée.	Reprendre un réseau complet ou un corrigé ; dissimuler du code repris, modifier les données de test ou fabriquer des mesures.

Assistants IA. Aide autorisée pour comprendre, documenter, déboguer et proposer du code ou des hypothèses. Déclarer les usages substantiels et les parties concernées. Tout code remis doit être compris, testé et défendable ; une sortie d'IA n'est ni une source scientifique ni une preuve. Aucun détecteur ne remplace la vérification individuelle.

6.1. Rendus

Un dépôt Git relançable contient sources, tests, configuration, graines, données/poids ou procédure exacte de génération, versions et mesures brutes CSV. Un manifeste relie chaque campagne au commit, au binaire et à la carte. Une autre équipe relance un cas. Une commande de tests PC et une CI GitHub Actions sont obligatoires ; elles ne certifient pas les performances de la carte.

Le **rapport contient au maximum 15 pages au total**, références et annexes comprises. Viser 6 à 10 pages utiles. Il présente : résumé, problème et sources, méthode, protocole, résultats, limites et suite. Inclure trois figures : apprentissage, réel/prédit et compromis matériel de l'optimisation. Ne pas imposer une amélioration positive. Un bilan individuel décrit la fonction et l'expérience maîtrisées. La soutenance individuelle comprend une explication et un petit changement prédit puis vérifié ; elle est organisée hors des 28 h. Chacun prédit puis vérifie une condition de mesure nouvelle annoncée à l'oral.

6.2. Barème sur 20

Collectif, 12 points : calculs/apprentissage corrects (3), déploiement et mesures (3), hypothèse et analyse des limites (3), DevOps, reproductibilité et clarté du compte rendu (3). Pour chaque critère : absent ou invalide, 0 ; partiel, moitié ; complet et vérifiable, totalité. **Individuel, 8 points :** expliquer une fonction et un résultat (4), prédire puis vérifier une modification (4). Un résultat négatif rigoureux peut obtenir tous les points.

Bonus de présentation : jusqu'à +2 points

+0,5 pour un rapport rédigé dans un anglais compréhensible ; +0,5 pour une présentation d'article scientifique soignée ; +1 pour un projet Overleaf gratuit recompilable avec ses sources, bibliographie et figures. **Note finale = min(20, note de base + bonus).** Aucun abonnement n'est requis. Les 20 points de base restent accessibles sans ces options ; la qualité scientifique et la traçabilité sont évaluées dans le socle.

Lire le document séparé *Consignes du rapport scientifique* et l'exemple anglais. Les annexes techniques volumineuses vont dans le dépôt, sans contourner la limite de pages. La date précise de décembre et la carte commune sont annoncées par l'enseignant avant le démarrage.

7. GitHub : rendre le travail visible et relançable

Dès le début : créer un dépôt GitHub par groupe, par exemple `inem-embedded-forecasting-g01`. Un dépôt privé convient. Inviter tous les membres et **rolln7drktayau** (enseignant) dans *Settings* → *Collaborators* → *Add people*. Vérifier l'acceptation avant le premier contrôle ; l'invitation seule ne prouve pas que l'accès fonctionne.

Le README donne les noms et comptes des membres, le lien du dépôt, le besoin étudié, la carte, les versions d'outils et les commandes exactes pour générer, tester, entraîner, compiler et exécuter sur Pi. **Les identifiants, fonctions, commentaires et messages de commit sont de préférence en anglais.** Expliquer les choix et les unités ; un commentaire qui répète une boucle ne suffit pas.

7.1. Premier envoi

Installer Git et s'authentifier selon sa documentation. Créer sur GitHub un dépôt vide. Dans votre dossier local contenant le README, adapter YOUR_ACCOUNT et gXX :

```
git init -b main
git add README.md
git commit -m "docs(project): describe objective and team"
git remote add origin https://github.com/YOUR_ACCOUNT/inem-embedded-forecasting-gXX.git
git push -u origin main
```

7.2. Une opération = un changement nommé et vérifiable

Créer une branche pour une tâche. Ajouter des commits cohérents et les pousser pendant le travail. Exemple, lorsque les dossiers `src` et `tests` existent :

```
git switch -c feat/dense-forward
git add src tests
git commit -m "feat(dense): implement scalar forward pass"
git push -u origin feat/dense-forward
```

Ouvrir une *pull request* vers `main`. Un autre membre relit, vérifie les tests et commente ; fusionner après correction. La branche principale reste utilisable. Puis récupérer la fusion et nommer l'étape validée :

```
git switch main
git pull --ff-only
git tag -a v0.2-core -m "Validated dense and gradient tests"
git push origin v0.2-core
```

Un commit enregistre localement ; `git push` envoie sur GitHub. Un tag désigne une version précise. Un push ne déploie pas le modèle sur le Pi : récupérer les fichiers, compiler et tester constituent une étape distincte.

7.3. Jalons et preuves attendues

Version poussée	État à retrouver dans GitHub
`v0.1-data`	Générateur, partition, statistiques train, figure des signaux et empreintes des CSV.
`v0.2-core`	Projection, activation, gradients et tests numériques qui passent.
`v0.3-trained`	Réseau, références, courbes, poids sauvegardés, configuration et rechargement vérifié.
`v0.4-pi-baseline`	Compilation sur Pi, équivalence PC/Pi et mesures de référence.
`v0.5-experiment`	Hypothèse datée, variante correcte, mesures brutes et analyse.
`v1.0-submission`	Version finale relançable : rapport, sources et bilan individuel.

Ces jalons sont des points de contrôle, pas un quota de commits. Conserver les étapes intermédiaires et les essais négatifs utiles. Ne pas reconstruire artificiellement l'historique à la fin. Exemples de branches : `feat/data-generator`, `feat/manual-backprop`, `feat/training-loop`, `feat/pi-inference`, `exp/loop-order`, `docs/paper`.

Organisation minimale à concevoir

```
README.md                # setup, commands, board, team
src/  tests/  configs/   # own model and automated checks
data/                    # small CSVs or exact generation recipe
artifacts/               # weights, normalization, checksums
experiments/             # hypothesis, manifests, raw CSVs
figures/  paper/         # plotting sources, PDF, LaTeX if used
.github/workflows/       # student-written CI workflow
```

Automatisation obligatoire. Écrire une commande de tests PC, puis une CI GitHub Actions qui la lance à chaque push et pull request. Tester au moins projection, gradients, sauvegarde/rechargement et un cas d'inférence connu. La CI prépare et vérifie le logiciel ; les temps du runner ne représentent pas les performances du Raspberry Pi.

Une mesure doit retrouver son origine

Chaque campagne possède un identifiant, le commit exact, une arborescence sans changements non enregistrés, les empreintes SHA-256 des données, des poids et du binaire, la configuration, la carte, la fréquence, le compilateur, ses options, le nombre d'appels et l'unité de mesure. Conserver un CSV brut, par exemple `campaign,variant,shape,repeat,elapsed_ns,calls,checksum`. Indiquer explicitement le périmètre chronométré.

Conserver les petites données nécessaires ou une recette exacte avec versions et empreintes vérifiées. Ne pas commiter mots de passe, jetons, environnements virtuels ou fichiers de construction volumineux. Si le binaire est déposé en pièce jointe de release, le relier au tag et donner son empreinte. Un résultat sans fichier brut ni configuration ne constitue pas une preuve reproductible.

8. Un dépôt organisé comme une petite étude scientifique

Une autre équipe doit pouvoir retrouver le code, les données, l'expérience et la figure correspondant à une conclusion. Séparer ces éléments évite de perdre les résultats ou de comparer deux configurations différentes.

```
README.md                    # question, auteurs, carte, commandes
src/                         # fonctions du modèle et apprentissage
deployment/                  # inférence et exécution sur Pi
tests/unit/                  # fonctions vérifiées séparément
tests/integration/           # chaîne et équivalence PC/carte
configs/                     # paramètres, graines, options
data/README.md               # origine, partitions, empreintes
artifacts/                   # poids et normalisation exportés
experiments/protocol.md       # hypothèse écrite avant les essais
experiments/raw/              # CSV originaux, jamais écrasés
experiments/manifests/        # une fiche par campagne
analysis/                    # scripts de figures et de tableaux
figures/                     # figures issues de ces scripts
paper/                       # PDF, sources article, bibliographie
docs/                        # décisions, limites, contributions, IA
.github/workflows/ci.yml      # tests et compilation automatiques
.gitignore                   # secrets et fichiers temporaires exclus
```

Ce que contient la fiche d'une expérience

Donner un identifiant unique, par exemple `dense-pair-16x8-run01`. La fiche précise la question, le commit, la variante, les dimensions, les données et poids avec empreintes, la cible, la fréquence, le compilateur et ses options, les répétitions, le périmètre chronométré, les unités et les chemins des CSV. Les modifications non enregistrées sont signalées ; pour la campagne finale, travailler sur une version propre.

Avant : « Nous pensons que calculer deux sorties ensemble réduit les lectures d'entrée. » Fixer le protocole et le critère de correction. **Pendant :** enregistrer chaque mesure, y compris les essais défavorables. **Après :** générer les figures depuis les CSV, expliquer la dispersion et proposer la prochaine expérience. Ne pas choisir uniquement le meilleur passage.

Exemple de commit : `perf(dense): group two outputs without changing weights`. Une expérience est reliée à ce commit, pas seulement au nom d'un fichier. Une erreur découverte donne lieu à un correctif et à de nouvelles mesures ; les anciennes traces sont conservées et marquées invalides.

9. Quels tests écrire, et que prouvent-ils ?

Un test vérifie une affirmation précise. « Le programme ne plante pas » ne prouve ni que le gradient est correct, ni que le modèle est utile. Écrire les tests au fur et à mesure ; conserver les premiers échecs utiles dans les discussions GitHub.

Niveau	Exemple et résultat attendu
Unitaire : une fonction	Projection sur un petit tableau calculé à la main ; ReLU sur valeurs négative, nulle et positive ; perte sur prédictions connues. Vérifier aussi les dimensions invalides.
Unitaire : dérivées	Différences finies pour toutes les familles de paramètres, sur deux entrées. Inclure les biais et les paramètres partagés ; respecter la tolérance du sujet.
Données	24 séquences et bons nombres de fenêtres. Modifier le test ne change pas les moyennes train. Une fenêtre ne traverse jamais une frontière de séquence.
Intégration PC	Sauvegarder puis relire les poids produit la même sortie. Sur quelques exemples, la perte diminue fortement : cela vérifie que l'apprentissage agit.
PC → carte	Les mêmes fenêtres, poids et statistiques produisent des sorties proches selon la tolérance. La carte calcule réellement la prédiction.
Variante optimisée	La brique modifiée reste correcte sur les formes du modèle, dont les sorties impaires. Puis vérifier le réseau entier avant de mesurer le temps.
Validation scientifique	Protocole figé, campagnes répétées, CSV disponibles, dispersion et effet global. La variante peut être correcte et néanmoins plus lente.

Automatisation et points de passage

Définir une commande unique pour les tests ; par exemple `python -m unittest discover -s tests -v` en Python, ou l'équivalent dans le langage choisi. La CI lance les tests et la compilation ; les tests matériels restent identifiés séparément. Une case ignorée faute de carte n'est pas une mesure réussie.

Prêt pour entraîner : données, dimensions et gradients vérifiés. **Prêt pour mesurer :** inférence PC/carte équivalente et timer vérifié. **Prêt pour conclure :** trois campagnes, résultats relançables et une reproduction par une autre équipe. Un test qui échoue se corrige avant de poursuivre.

À la soutenance, chacun explique une fonction et prédit l'effet d'une modification simple avant de la tester. Les contributions, revues et expériences rendent la compréhension observable ; aucune méthode ne garantit l'absence de triche.

10. Outils et sources : quoi télécharger ?

Il n'y a aucun modèle préentraîné ni jeu de données à télécharger. Vous générez les signaux de la section 2 et entraînez vos 388 paramètres. Aucun capteur, GPU, compte cloud payant ou environnement fourni n'est nécessaire. Le matériel minimal par groupe est un PC, un Raspberry Pi alimenté, une microSD avec OS et un accès réseau ou écran/clavier.

Outil ou ressource officielle	Usage et lien
Git et GitHub	git-scm.com/downloads ; installation et authentification ; inviter les collaborateurs.
Python, si choisi sur PC	python.org/downloads. random pour les tirages, csv pour les fichiers ; standard math pour sinus, cosinus et racine. Noter la version et le générateur.
C/C++ ou Rust	GCC ; Rust. Compiler sur le Pi avec les mêmes options pour référence et variante ; noter la version.
Outils de la carte	Raspberry Pi Imager et installation officielle ; SSH et transfert. Installer Git et le compilateur choisi sur le Pi ; tester une exécution avant TD3.
Figures et CI	Matplotlib, autorisé pour les figures ; GitHub Actions, pour créer votre automatisation. Pas de NumPy dans le modèle évalué.
Article et LaTeX	Overleaf ; premier projet et import ZIP ; classe elsarticle. Le ZIP fourni contient un gabarit et un exemple, pas une solution du TD.

Lectures scientifiques : comprendre, pas recopier

[1] Chen et al., TSMixer (2023) et code des auteurs : architecture complète, distincte de notre petit réseau.
[2] Google Research : présentation illustrée de TSMixer.
[3] Stanford CS231n : contrôle des gradients.
[4] OpenBLAS : bibliothèque optimisée, uniquement comme comparateur séparé ; comparer un calcul et un nombre de threads équivalents.
[5] Jacob et al. (2018) : calcul entier et quantification, extension facultative.

Première action concrète : vérifier le modèle de carte et les outils, créer le dépôt, inviter l'enseignant, écrire le générateur, afficher une séquence et expliquer la cible à prédire. Ne commencer l'optimisation qu'après les tests et la référence sur carte.

Petit lexique. CPU = processeur de calcul ; RAM = mémoire de travail ; microSD = stockage conservé hors alimentation. OS = système d'exploitation ; SSH = accès distant au terminal. RSS = mémoire physique occupée par un processus ; un buffer est un tableau temporaire. Le cache rapproche des données du CPU. CI = intégration continue ; CSV = fichier de valeurs séparées par des virgules. SHA-256 produit une empreinte qui permet de vérifier qu'un fichier n'a pas changé.